

Introduction

Of the many things Intune is good at, you will not find reporting listed. This is because the design phase at Microsoft, like all large corporate IT projects, did not take compliance monitoring and audit requirements seriously. They have an extensive list of configuration settings and compliance policies, but getting that data in a meaningful form from the Intune console can be a challenge.

There is a common perception that console snapshots can provide enough evidence and that if there is a requirement for better or more detailed evidence that you can push back to the auditor and they will go away. In fact, the opposite always happens, an audit finding gets opened and you now have no choice but to find a way to get that evidence.

Key drivers

“Trust but verify” (Zero Trust)

A good example is Bitlocker where you can require bitlocker encryption on all windows devices, see in the console that you have X% compliant and non-compliant devices, but can't prove why they are compliant or not. It can be difficult to determine if the drive protector is on, if the bitlocker service is running. These are some examples of the many other Key Compliance Indicators (KCI's) an auditor might ask for evidence of.

If you have a “trust but verify” approach, then you need to collect and provide details that support the X% compliant calculation. You never let the fox watch the henhouse.

Impact on Source and vice versa (Throttling and Security)

Your reporting using Graph API has an impact felt across your tenant. If you are using SharePoint, One Drive and Teams, are onboarding and offboarding devices, and generally just using the product portals, all this traffic uses the Graph API at one point or another.

Each team must work to have a minimal impact on Graph API usage due to the tenant restriction of 130,000 requests per 10 seconds. This may not sound significant, but when you have 70,000 users and 300,000 devices, you can quickly hit that throttling wall querying configuration settings and device compliance status, especially if other support teams are performing a migration or permissions audit.

The best solution is to replicate the data to a “sandbox” once to isolate the golden source from the impact of your team’s activities and then your ETL processes and reporting analysts can perform as many queries as they want without impacting other teams’ functionality.

This also has the added benefit of increasing your security posture in that none of the data consumers have any permissions in the Graph API. All you need to do is make sure the replication process uses an App Registration, replicate the data to the “sandbox” and control access to the “sandbox.”

Data volumes and Time constraints for report production (ETL Processes)

I saw a demonstration using Power BI to produce a compliance dashboard. It was pretty responsive in the lab with 4 devices. Scaling it up showed its limitations. Performing ETL processes using Power Query is not very efficient. Filtering data at the source is an option, collecting just what you need, but you may not get all your dashboards updated in time for the data consumers delivery requirements.

As an example, each report may have several requirements in common with other reports. Most of my reports contain inventory data such as device name, serial, location (to the city) as well as business information for device ownership. If this process takes one hour to create, and is used by 8 reports, then by saving this common data and re-using it for the other reports can save 7 hours of effort overall to produce the 8 reports.

The SQL “sandbox” ingesting the same filtered data can normalize and index the result data for speed, combine repeated actions to create derived sources for re-use, and dimension the data to reduce required storage thus optimizing the data pipeline to produce results faster than Power Query.

Audit and enterprise visibility (SDLC and segregation of environments)

Auditors and Cybersecurity groups often demand compliance statistics for all machines in the environment. This includes the DEV and UAT tenants as well as the Production tenant and requires combining the data across these sources to provide a unified view of device coverage.

For instance, the agent for an intrusion detection system or vulnerability scanning needs to be deployed to every system regardless of environment and operating system. Without a method to verify their proper deployment and operation, you could be exposed and never know.

Historical evidence retention requirements (Audit requirement)

Data in Azure and Intune have a limited lifetime, often 30 days. Auditors may require that the compliance data you generate persist for up to a year. Having a “sandbox” allows you to meet that requirement easily.

Components

PostgreSQL, the “sandbox”

Postgres would not have been my first choice for a data storage and ETL platform. Normally, I would use a SQL server (Microsoft SQL Server...) that supports some sort of integrated authentication. Since my SQL server is in the same local network, but not in an Active Directory infrastructure, and is not technically part of my lab (Azure joined), I am forced, no matter what product I use, to settle for username and password authentication. It is a security risk, but it is a lab.

The bonus for Postgres is that it is different than what I would normally use and there are skills and learning points I can gain before I use it as it should be used. There is a lot of interest in the use of Postgres on the cloud and more especially for AI model storage and retrieval.

Powershell: ODBC Driver download: [PostgreSQL: File Browser](#)

App Registration (Service Principal)

Creating and securing an app registration has already been covered in one of my previous projects. I will need to port some of that over to this document and then create a Powershell script to create a subordinate certificate using an existing self-signed trusted root CA.

Data Collection Components

I already have the JWT assertion and token acquisition in one project, and unauthenticated REST queries in another. Again, this knowledge needs to be combined, but not invented to form a collection of templates for getting what we need in terms of data.

ETL Processes

This is where the fun begins, the “brain dump”.

Process Orchestration

Here is where the experimentation begins. There is a pgAgent I can use, and I already built an orchestration platform once in C#. I want to avoid adding this complexity to the project if I can.

PGAdmin?

REST as a data source

Install and configure Postgres

<Documented in Postgres folder>

Create client certificate

Certificates, trust chains, CA signed and Self-signed certificates is too long a discussion for me to have and is not the scope of this document.

I have a “Lab Root CA” self signed trusted root deployed in my lab with signed client certificates created, signed and deployed for various uses. That may be more complexity than you want.

You can find the Powershell script to create them in my previous lab here:

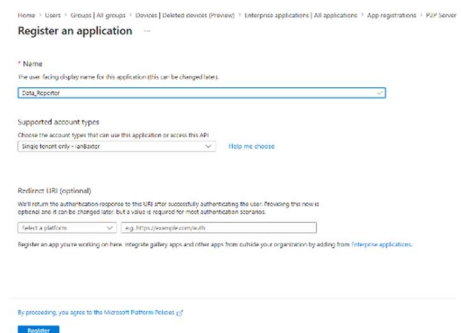
If you just need to create a client certificate, CA signed or self-signed, I have another script in this project – “Client_Cert.ps1”.

You will find all certificates generated in your Documents folder.

Create App Registration

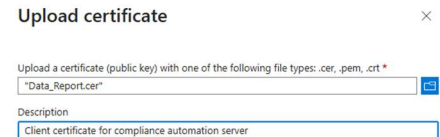
Give it a descriptive name.

Make sure it is set for single tenant.



The screenshot shows the 'Register an application' page in the Microsoft Azure portal. The breadcrumb navigation at the top reads: Home > Users > Groups > All groups > Devices > Deleted devices (previous) > Enterprise applications > All applications > App registrations > POP Server. The page title is 'Register an application'. There is a 'Name' field with the value 'Data Reporter' and a note that the user-facing display name can be changed later. Below this is the 'Supported account types' section, where 'Choose the account types that can use this application or across this app' is set to 'Single tenant only - administrator', with a 'Help me choose' link. The 'Redirect URI (optional)' section has a note about returning the authentication response and a 'Select a platform' dropdown menu with 'Web API (https://example.com/api)' selected. At the bottom, there is a 'By proceeding, you agree to the Microsoft Platform Policies' checkbox and a 'Register' button.

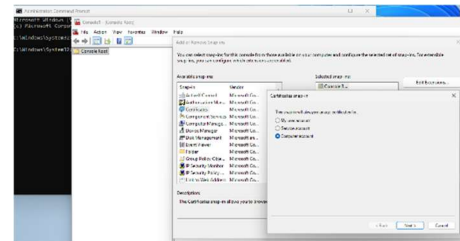
Select “Certificates and Secrets”, select the “Certificates” tab and upload your client certificate.



As we dig into Graph API and the other REST calls, we will add application permissions.

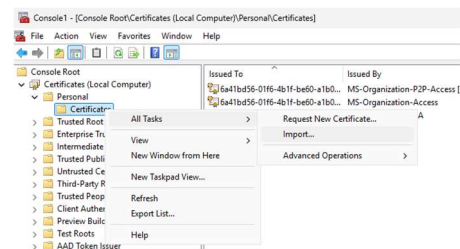
Install the client certificate (Windows)

Find “CMD” in the start menu, right-click on it and chose “Run as Administrator”. Using the command prompt, run “MM”, add the “Certificates” snap-in and choose “Computer Account”.

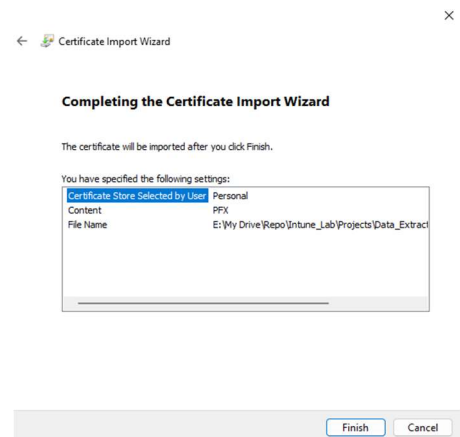


Select “Local Computer” and click “Finish”, then click “OK”.

Expand the menu down to the personal store, right-click on “Certificates”, “All Tasks” and then “Import”.

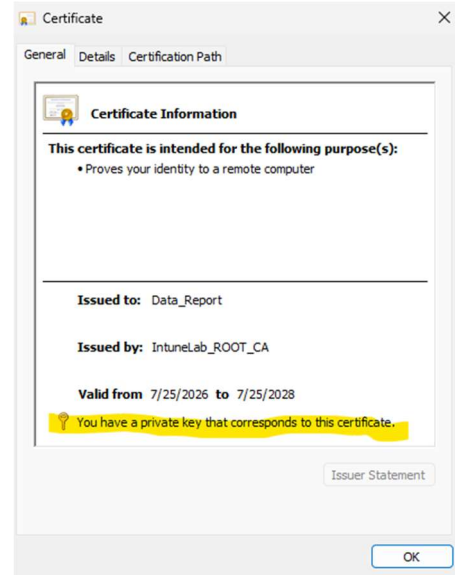


Find your .pfx file and upload it, enter the password and complete the import.

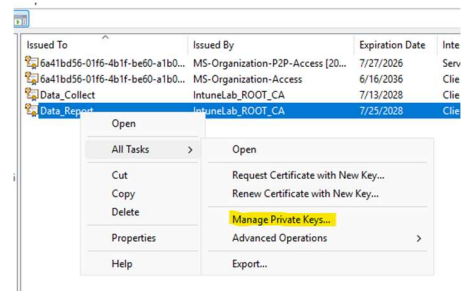


Open the certificate by double-clicking on it.

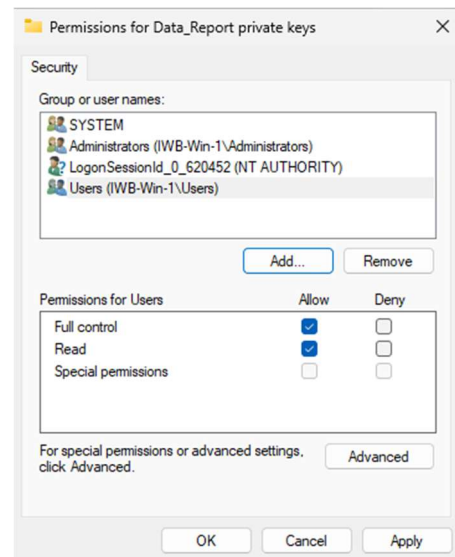
Verify it has a private key.



Close the window, right-click on the certificate, choose “All Tasks” and “Manage Private Keys”.



Grant “Read” and “Full Control” to the Users group.



Script – Entra ID Users (List query)

The first bookmark you should save is: [Microsoft Graph REST API beta endpoint reference - Microsoft Graph beta | Microsoft Learn](#)

I use the beta API because the V1.0 API doesn't have all the data I need. The beta API is closer and seems generally safe to use.

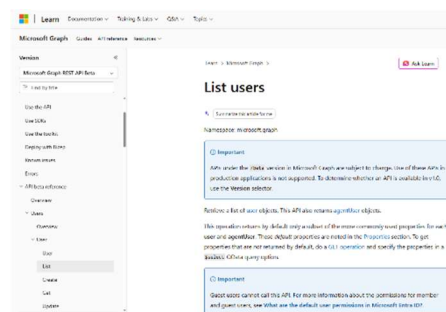
The next API you should save is: [Graph Explorer | Try Microsoft Graph APIs - Microsoft Graph](#)

This online tool will enable you to try your REST calls and view sample returned data. This is critical to modify the data ingestion code for your own purposes.

When you start a journey into Graph API, it's best to start with an easy example, a simple LIST query. This one will get us all the user information for this tenant.

The API call we will use is documented here: [List users - Microsoft Graph beta | Microsoft Learn](#)

I suggest reading the information in this link. It will provide many samples of queries, including filtering and searching to reduce the volumes of data returned



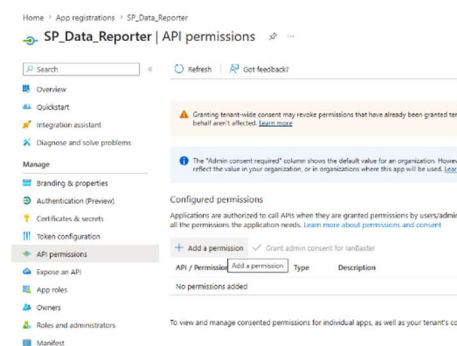
It lists what permissions are required to query the data.

We are only concerned with application permissions that are least privilege.

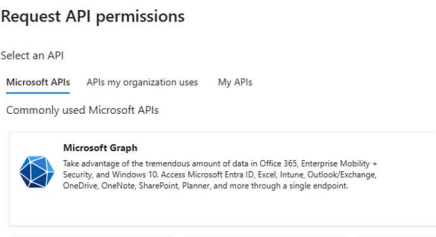
Permission type	Least privileged permission	Higher privileged permissions
Delegated (work or school account)	User.ReadBasic.All	User.Read.All, User.ReadWrite.All, Directory.Read.All, Directory.ReadWrite.All
Delegated (personal Microsoft account)	Not supported.	Not supported.
Application	User.Read.All	User.ReadWrite.All, Directory.Read.All, Directory.ReadWrite.All

Paging is a critical in that it can be used to reduce the number of Graph API calls to query the entire list. By default, you get 100 user accounts per page of data, and each page takes a Graph API call. By increasing this to 500, we can reduce the number of API calls by a factor of 5 in large environment. Each paged response will have a property named “@odata.nextLink” which will have the URL to use to get the next page of data, or null if you are at the end of the list.

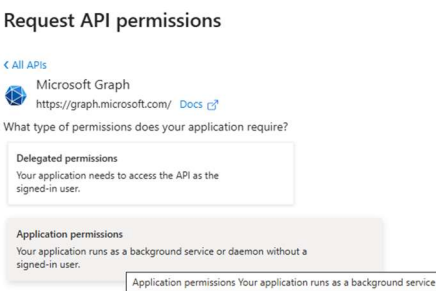
Before we examine the script, your App Registration will need the necessary API permissions. In the Entra ID portal, locate your App Registration and click on it. Choose “API Permissions” in the left pane and click “+ Add a permission”.



Select “Graph API”.



Select “Application permissions”.



Add Directory.Read.All and User.Read.All

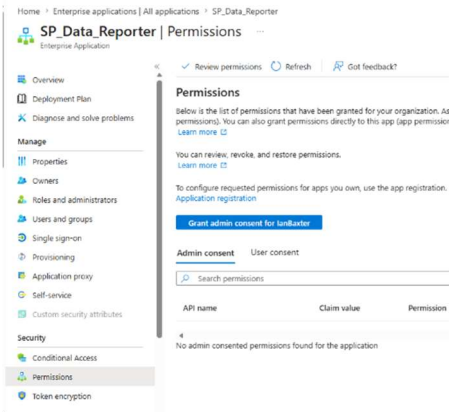
+ Add a permission

✓ Grant admin consent for lanBaxter

API / Permissions name	Type	Description	Admin consent requ...	Status
Microsoft Graph (2)				
Directory.Read.All	Application	Read directory data	Yes	⚠ Not granted for lanBaxter
User.Read.All	Application	Read all users' full profiles	Yes	⚠ Not granted for lanBaxter

You now need to grant admin consent for these permissions. In the Entra ID portal, select “Enterprise apps”, locate your App Registration and click on it.

Click on “Permissions” in the left pane and then click on the “Grant admin consent...” button.



Going back to the App Registration entry, you can verify admin consent has been granted and the assigned permissions can now be used.

API / Permissions name	Type	Description	Admin consent requ...	Status
Microsoft Graph (2)				
Directory.Read.All	Application	Read directory data	Yes	✔ Granted for lanBaxter
User.Read.All	Application	Read all users' full profiles	Yes	✔ Granted for lanBaxter

Script – Entra ID Devices

Script – Entra ID groups

Script – Intune Managed Devices